Assignment 1: Report

Martina Carminati, Sanna Plomp, Keval Mehta

November 7, 2026

1 Learning the Basics of PyTorch

1.1 Multi-layer Perceptron

In this first part, we started by setting a seed for reproducibility, and then importing the Fashion-MNIST dataset from `torchvision.datasets`. The training and test datasets were imported separately, and the training dataset was split into two subsets: what we will use for training (90%), and validation (10%). Setting batch size to 64, we created data loaders that will then load the data into the models.

To start off the systematic experimentation, we used a baseline model with the following parameters: `hidden_size` $=$ 256, `dropout` $=$ 0.2, `optimiser` $=$ SGD, and `learning_rate` $=$ 0.01. For validation, 10 epochs were used to get a good feel for the accuracy without overfitting. The experimentation is done sequentially with a small amount of values for each hyperparameter, more as a show of proof than to find the all-time highest possible accuracy.

In the first round, only the activation function was varied, while keeping the other parameters fixed. The following activation functions were used.

ReLU

$$f(x) = \begin{cases} x & x \geqslant 0 \\ 0 & \text{otherwise} \end{cases},$$

Leaky ReLU

$$f(x) = \begin{cases} x & x \geqslant 0 \\ mx & \text{otherwise} \end{cases},$$

GeLU

$$f(x) = x \cdot \Phi(x).$$

Leaky ReLU resulted in the highest validation accuracy of 0.8857. See Table 1 for the results of the different activation functions.

Activation	Optimiser	Validation Accuracy
ReLU	SGD	88.32%
Leaky ReLU		88.57%
GELU		87.73%

Table 1: Performance on validation dataset by different activation functions

Next, we will continue varying the optimisers, using ReLU as the activation function. All other hyperparameters were kept fixed, as SGD, Adam, and AdamW were tested. Adam resulted in the highest validation accuracy $=$ 0.8568. See Table 2 for the results of the different optimisers.

Continuing, we varied the learning rate. The following values were used for the learning rate [0.001, 0.01, 0.1]. Learning rate $=$ 0.001 resulted in the highest validation accuracy of 0.8872.

The size of the hidden layer was also varied. The following hidden sizes were tested: 128, 256, and 512. From the results, the hidden size of 256 was the best, with a validation accuracy of 0.8560.

Activation	Optimiser	Validation Accuracy
Leaky ReLU	SGD	84.77%
	Adam	86.58%
	AdamW	83.08%

Table 2: Performance on validation dataset by different optimisers

Finally, we tried out dropout values of 0.0, 0.2, and 0.5. Dropout $= 0.0$ resulted in the highest validation accuracy of 0.8925. The top 3 performing models were then retrained on the train set, and tested on the test set. The results are shown in Table 3. The best performing model was the one with Leaky ReLU, Adam, learning rate $= 0.001$, hidden size $= 256$, and dropout $= 0.0$, with a test accuracy of 0.8881.

Activation	Hidden Size	Dropout	Optimiser	Learning Rate	Test Accuracy
Leaky ReLU	256	0.0	Adam	0.001	88.81%
	256	0.2	Adam	0.001	88.29%
	256	0.2	SGD	0.01	83.24%

Table 3: Performance on Fashion-MNIST test set by different configurations

These top 3 models were then trained and tested on the CIFAR-10 dataset, adjusting them only for number of channels. Performance was not as strong, as demonstrated in Table 4. The best performing model was the one with Leaky ReLU, Adam, learning rate $= 0.001$, hidden size $= 256$, and dropout $= 0.0$, with a test accuracy of 0.4842.

Activation	Hidden Size	Dropout	Optimiser	Learning Rate	Test Accuracy
Leaky ReLU	256	0.0	Adam	0.001	48.42%
	256	0.2	Adam	0.001	43.54%
	256	0.2	SGD	0.01	44.02%

Table 4: Performance on CIFAR-10 test set by different configurations

The multi-layer perceptron model works by assigning weights to pixels, in comparison to convolutional structures, which takes into account neighbouring pixel values as well. This means that the network tries to learn weights by pixel, which is a strength for values that are invariate, applying roughly to the Fashion-MNIST dataset as images are centred and approximately at the same location, as compared to CIFAR-10, where the images' constituents move around within the images, and aren't always at the same location, which doesn't lead to a high accuracy.

Adding more layers and giving the network a capability to be more flexible would help increase accuracy, but the point remains that MLPs aren't the best tools for images.

Within the three results, a dropout of 0.2 does worse than no dropout, which may be because MLP needs all its capacity to perform, and dropout randomly sets some activations to 0 to prevent overfitting, which is not happening in this case, and hence dropout does not help performance.

1.2 Convolutional Neural Networks

Like in the previous section, a seed was set for reproducibility, and the dataset was imported. After importing the train and test separately, we split the training dataset into two subsets: what we will use for training (90%), and validation (10%). Setting batch size to 64, we created data loaders that will then load the data into the models.

We set up a basic convolutional neural network, similar to the one depicted in Figure 1, with two convolutional layers and max pooling between, with a customisable activation function.

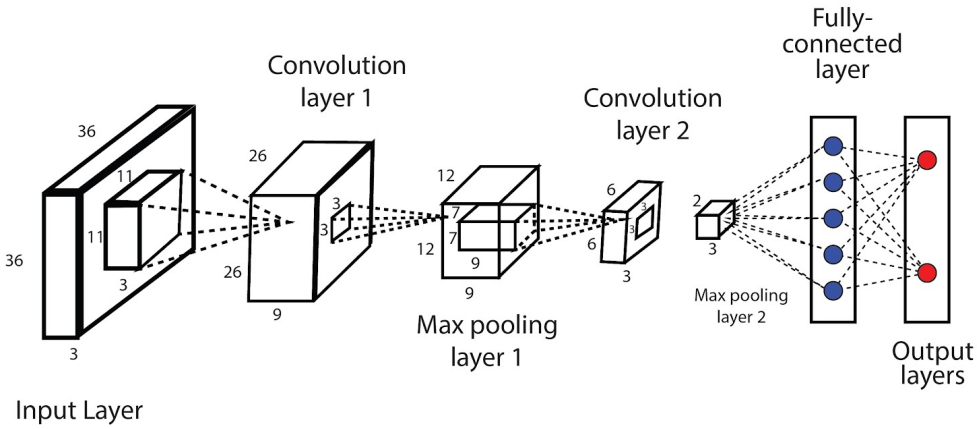


Figure 1: Sample Convolutional Neural Network

Defining functions for training, validating, and testing, we then moved to the systematic testing of parameters.
We performed this testing with 10 epochs for training, which was validated through the separated dataset. The test performance of that model was then calculated. The performance of the different activation functions is given in Table 5. As we can see in the table, GELU performed best, so we will use it to fit the rest.

Activation	**Optimiser**	**Validation Accuracy**
ReLU	SGD	81.25%
Leaky ReLU		81.12%
GELU		81.75%

Table 5: Performance on validation dataset by different activation functions

Activation	**Optimiser**	**Validation Accuracy**
GELU	SGD	81.75%
	Adam	90.13%
	AdamW	90.00%

Table 6: Performance on validation dataset by different optimisers

From Table 6, we can see Adam does best, so we will use that in our further analysis. Now, we test different parameter combinations of the optimiser we have chosen, and the results are present in 7. We can see that the best performing model, as per our combined analysis, is with the GELU activation function, Adam optimiser, learning rate$= 10^{-3}$, weight decay$= 10^{-5}$, and betas$= (0.9, 0.999)$. We see a consistently strong performance with these values of learning rate and betas, even when weight decay is varied, even though that does decrease it marginally.

Activation	Optimiser	Validation Accuracy	Learning Rate	Weight Decay	Betas
GELU	Adam	0.9035	10^{-3}	10^{-5}	$(0.9, 0.999)$
		0.9002	10^{-3}	10^{-5}	$(0.95, 0.995)$
		0.8960	10^{-3}	10^{-5}	$(0.99, 0.99)$
		0.9025	10^{-3}	10^{-4}	$(0.9, 0.999)$
		0.8948	10^{-3}	10^{-4}	$(0.95, 0.995)$
		0.8888	10^{-3}	10^{-4}	$(0.99, 0.99)$
		0.8837	10^{-2}	10^{-5}	$(0.9, 0.999)$
		0.8787	10^{-2}	10^{-5}	$(0.95, 0.995)$
		0.8878	10^{-2}	10^{-5}	$(0.99, 0.99)$
		0.8800	10^{-2}	10^{-4}	$(0.9, 0.999)$
		0.8623	10^{-2}	10^{-4}	$(0.95, 0.995)$
		0.8803	10^{-2}	10^{-4}	$(0.99, 0.99)$

Table 7: Performance on validation dataset by different parameters

Further, after this, we ran the models on the CIFAR-10 dataset, adjusting them only for number of channels. Performance was not as strong, as demonstrated in Table 8. After running the three best models from Table 7, we tested different optimisers as well, to see more examples, as shown in Table 9. None of these performed particularly well.

Learning Rate	Weight Decay	Betas	Test Accuracy
10^{-3}	10^{-5}	$(0.9, 0.999)$	0.6216
10^{-3}	10^{-4}	$(0.9, 0.999)$	0.6172
10^{-3}	10^{-5}	$(0.95, 0.995)$	0.6407

Table 8: CIFAR-10 performance on the best three parameter combinations of `optim.Adam`

Name	Test Accuracy
Adam	0.6288
AdamW	0.6289
SGD	0.4495

Table 9: CIFAR-10 performance on different optimisers

This could be due to the fact that in essence, Fashion-MNIST is a much simpler task, where the items of clothing are centred, and single-channel, and there is possibly a high level of separation between classes. The images of CIFAR-10 are not only RGB, but more complicated, and not as well separated. SGD performs much worse than Adam, which could be due to a weakly-trained learning rate on CIFAR, as opposed to Adam, which generalises better and isn't as dependent on the learning rate, as it uses the moments to adapt. While this difference is large, there is not much between Adam and AdamW, which may be due to both not overfitting, hence the decay is not a huge advantage.

Within the Adam optimiser, lowering the decay enables it to adapt faster, which may make it perform better on a complex dataset. I was not able to vary the architecture as much as I would've liked to, but adding more convolutional kernels may help the models to generalise better to the more complicated CIFAR-10 dataset.

2 Develop a “Tell-the-Time” Network

For the second task, we had to design, train, and evaluate a convolutional neural network on images of clocks to tell the time using the learnings from Task 1.

2.1 Classification

2.2 Regression

2.3 Multi-Head Models